MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gherasim Sergiu FAF-242

Report

Practical Work No. 4

Computer Systems

Topic: The xv6 Shell from the Outside

Verified by: Reițman Patricia

Chișinău – 2026

Part 1 3
Part 2 4
1. List the features both shells have, and the ones only bash has. 6
Part 3 8
xv6 9
Part 4 9
Conclusion 11

Part 1

In this part we need to rename the sleep system call to pause. However, since I already did it in the second lab, I am just going to show a screenshot of how I changed it and prove that it works.

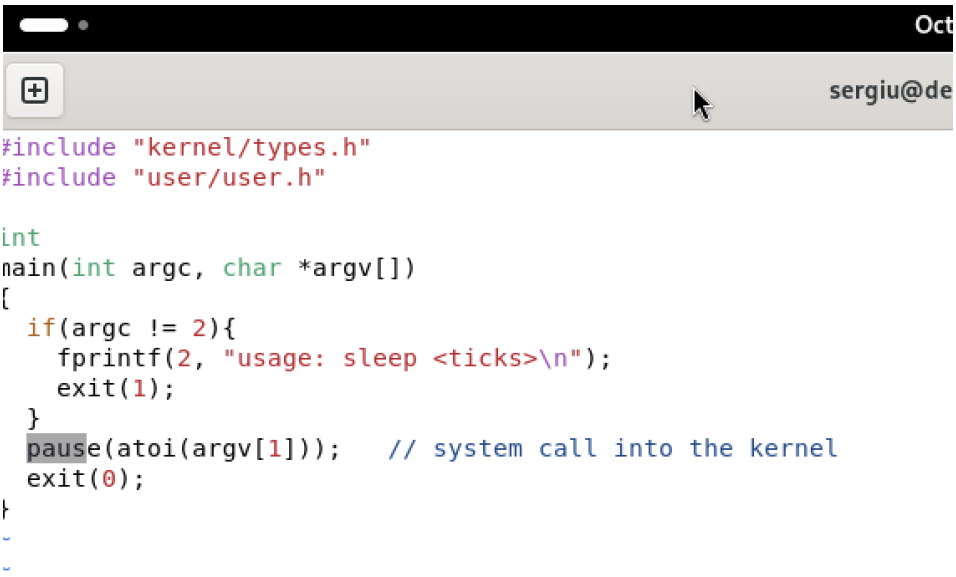


Figure 1.1 - The pause kernel system call being used in sleep.c

As you might have noticed, I didn't rename the sleep program in /user. I instead replaced the sleep system call with the pause system call.

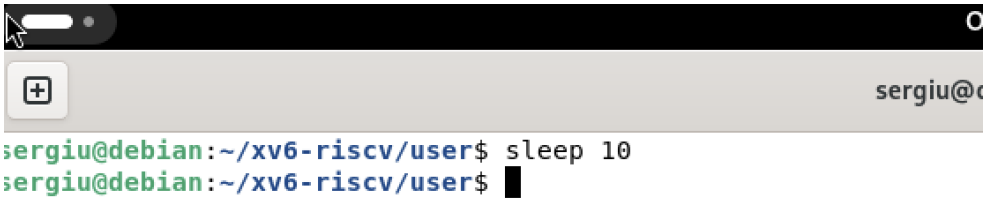


Figure 1.2 - The sleep app working

Part 2

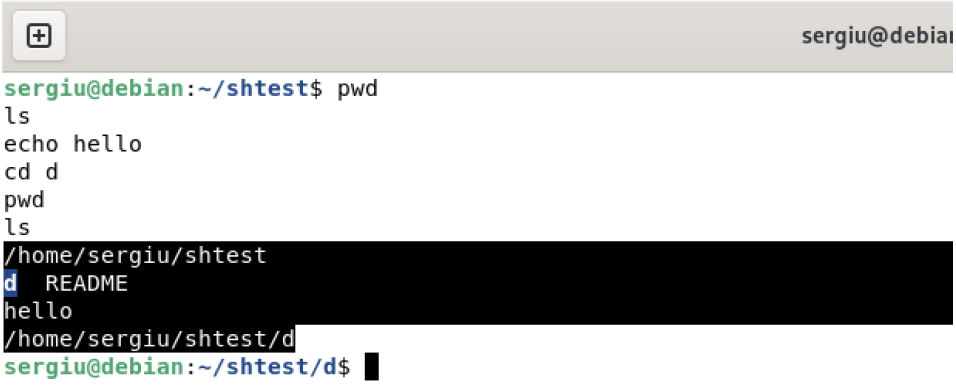


Figure 2.1 - The sleep app working

Command	**Tests**	**bash**	**xv6**
echo hello > f ; cat < f	redirection	Works	Works
ls | grep READ	pipe	Works – echos README	Works – but echos differently. Doesn't just print README but also the *name*, *type*, *inode* and *type*
(echo a; echo b) | wc	grouping, list	Echos numbers	Echos "syntax". xv6 doesn't support multiline command entry like bash
echo one ; echo two	sequence	Echos "one \n two"	Echos "one \n two"
echo bg &	background	[1] 4877	bg
ls *	globbing	f README d:	ls: cannot open *
echo $HOME	variables	/home/sergiu	$HOME
echo "a b"	quotes	a b	"a b" (xv6 doesn't recognize double quotes)
mkdir d ; cd d ; ls ..	PATH	d f README	exec mkdir failed exec cd failed . 1 1 1024 .. 1 1 1024 README 2 2 2441 cat 2 3 36728 echo 2 4 35560 forktest 2 5 17464 grep 2 6 40176 init 2 7 36016 kill 2 8 35496 ln 2 9 35312 ls 2 10 38856 mkdir 2 11 35544 rm 2 12 35528 sh 2 13 58608 stressfs 2 14 36416 usertests 2 15 202984 grind 2 16 51792 wc 2 17 37632 zombie 2 18 34896 logstress 2 19 37480 forphan 2 20 36344 dorphan 2 21 35808 sync 2 22 34960 sleep 2 23 35280 console 3 24 0 f 2 26 6
arrow up, Tab, Ctrl+R	history, completion	Work	Don't work

1. **List the features both shells have, and the ones only bash has.**

Both have:

- Command execution
- Pipes (|)
- Input/output redirection (<, >)
- Background execution (&)
- Command grouping (())
- Sequential execution
- Directory navigation (cd)

Only bash has:

- Command history
- Tab completion
- Environment variables
- PATH lookup
- Job control - jobs, fg, and bg
- Advanced scripting – variables, loops, etc
- Command substitution
- Advanced command grouping

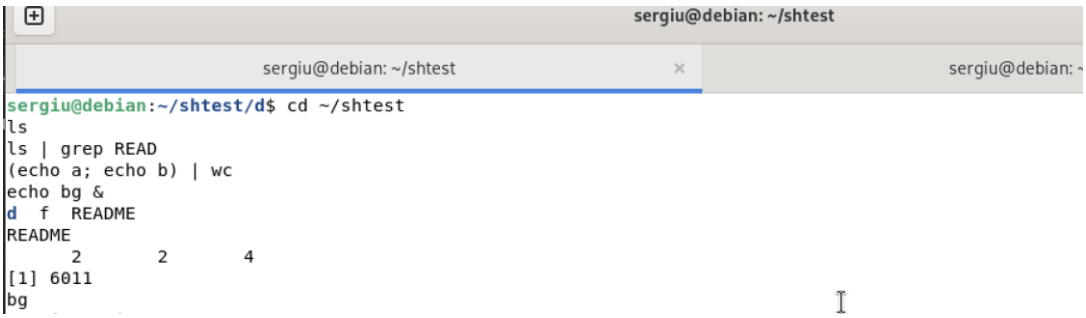


Figure 2.1 - The commands run in bash

2. **In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway.**

(Hint: where do the xv6 programs live?)
ls doesn't work inside d because we are not in the root directory. We run it from the d by specifying the absolute path for /ls.

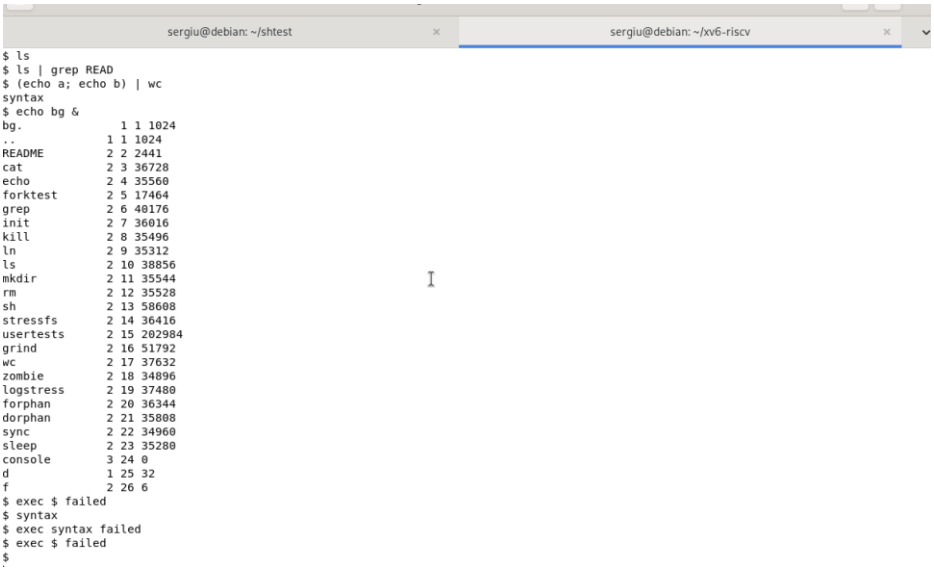


Figure 2.2 - The same commands run in xv6

3. Which missing feature would you miss most in daily work, and which would be easiest to add?

I would probably miss command history and tab completions as I use them the most. The easiest to add I think would be command history as it could just be a file that lists all of the commands entered in the current session.



Figure 2.3 - ls failing after cd d, and the working alternative

Part 3



Figure 3.1 - a args.c and the UPROGS line

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ args one two three
$ args *.c "x y"
$ args $HOME
$ args 'single quoted'argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ exec $ failed
$ exec $ failed
$
```

Figure 3.2 - args output in xv6.

```
sergiu@debian: ~/shtest                    sergiu@debian: ~/shtest
sergiu@debian:~$ cd ~/shtest
sergiu@debian:~/shtest$ vim args_linux.c
sergiu@debian:~/shtest$ gcc -o args args_linux.c
sergiu@debian:~/shtest$ ./args one two three
./args *.c "x y"
./args $HOME
./args 'single quoted'
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
argc = 2
argv[0] = [./args]
argv[1] = [/home/sergiu]
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 - args_linux output in bash

1. Paste both outputs for args *.c "x y". How many arguments did each program receive?

Bash

```
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
```

xv6

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

Bash receives 3 arguments, while xv6 receives 4.

2. So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?

The shell is responsible for expanding *.c into matching filenames and removing quotes before passing arguments to the program. In bash, wildcard expansion and quote removal happen automatically. In xv6, the shell does not support these features, so the wildcard and quotes are passed as literal characters.

3. What happened to the extra spaces in one two three?

The shell treats consecutive unquoted spaces as argument separators and discards the extra spaces. Therefore, one two three is passed as three separate arguments, regardless of how many spaces are between the words. Both bash and xv6 behave similarly in this case.

Part 4

1. Name the five kinds of command the xv6 shell knows and the character that produces each.

EXEC (normal commands), REDIR (<, >, >>), PIPE (|), LIST (;), and BACK (&).

```
sergiu@debian:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
sergiu@debian:~/xv6-riscv$
```

Figure 4.1 - The command kinds defined in sh.c

2. Where is cd handled, and why is it not run as a separate program?

cd is handled directly in user/sh.c as a built-in command. It must run in the shell process because a separate program would only change its own working directory, not the shell's.

```
  if (fork1() == 0) {
    close(1);
    dup(p[1]);
    close(p[0]);
    close(p[1]);
    runcmd(pcmd->left);
  }
  if (fork1() == 0) {
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    runcmd(pcmd->right);
  }
  close(p[0]);
  close(p[1]);
  wait(0);
  wait(0);
  break;

case BACK:
  bcmd = (struct backcmd *)cmd;
  if (fork1() == 0)
    runcmd(bcmd->cmd);
  break;
}
  exit(0);
}

int
getcmd(char *buf, int nbuf)
{
  write(2, "$ ", 2);
  memset(buf, 0, nbuf);
  gets(buf, nbuf);
  if (buf[0] == 0) // EOF
sergiu@debian:~/xv6-riscv$
```

Figure 4.2 - Where each kind is run

3. Search sh.c for any code that handles * or $. What does that explain about Part 2?

Wildcard (*) and variable ($) expansion are not implemented in sh.c. This explains why xv6 treats them as literal characters instead of expanding them like bash does.

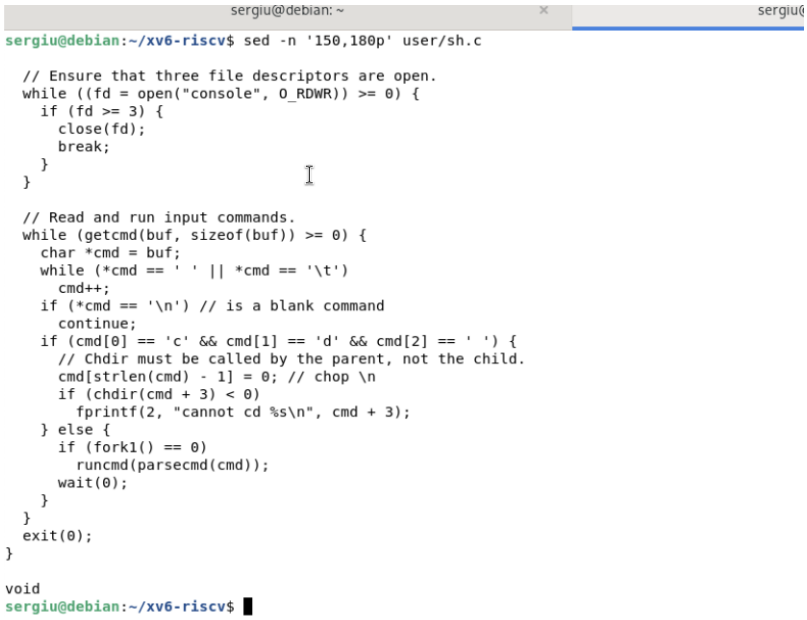


Figure 4.3 - The code that handles cd

Conclusion

In this practical work, I explored the functionality of the xv6 shell and compared it with the Linux bash shell. I learned how both shells handle command execution, pipes, redirection, background processes, and directory navigation. By testing the same commands in both environments, I observed that xv6 provides basic shell functionality but lacks advanced features such as command history, tab completion, wildcard expansion, and environment variables.

I also created a C program to examine how command-line arguments are processed and discovered that the shell, rather than the kernel or the program itself, is responsible for interpreting wildcards and quotes. Finally, by examining the source code of sh.c, I gained a better understanding of how the xv6 shell parses and executes commands and why cd must be implemented as a built-in command.

Overall, this laboratory helped me better understand how shells work internally and appreciate the additional functionality provided by modern shells such as bash.